


```

when DN_INDEX => declare NAME: constant TREE := D ( AS_NAME, NODE ); begin WALK ( NAME, NODE, REGION );
D( SM_TYPE_SPEC, NODE, D ( SM_TYPE_SPEC, D ( SM_DEFN, NAME ) ) ); D( AS_NAME, NODE, MAKE_USED_NAME_ID ( LX_SRCPOS, N
AME ), LX_SYMREP => D ( LX_SYMREP, NAME ), SM_DEFN, NAME ); end; when others => declare
ITEM_LIST: SEQ_TYPE; ITEM_NODE: TREE; begin case ARITY ( NODE ) is when NULLARY => null; when UNARY =>
WALK ( SON_1 ( NODE ), NODE, REGION ); when BINARY => WALK ( SON_1 ( NODE ), NODE, REGION ); WALK ( SON_2 ( NODE ), NODE, REGION ); whe
n TERNARY => WALK ( SON_1 ( NODE ), NODE, REGION ); WALK ( SON_2 ( NODE ), NODE, REGION ); WALK ( SON_3 ( NODE ), NODE, REGION ); wh
en ARBITRARY => ITEM_LIST := LIST ( NODE ); while not IS_EMPTY ( ITEM_LIST ) loop POP ( ITEM_LIST, ITEM_NODE ); WALK ( I
TEM_NODE, NODE, REGION ); end loop; end case; end;
0130 ut SEQ_TYPE ) is use PRENAME; NEW_ID_LIST: SEQ_TYPE := ( TREE_NIL, TREE_NIL ); procedure MAKE_PREDEF_IDS ( ID_LIST : o
ut SEQ_TYPE; NEW_ARG: TREE; ITEM_LENGTH: NATURAL; use MAKE_NOD; begin for PRAGMA_NAME in DEFINED_PRAGMAS l
oop NEW_ARG_LIST := ( TREE_NIL, TREE_NIL ); if PRAGMA_NAME = LIST or PRAGMA_NAME = PRENAME.DEBUG then for ARG_NAME in LIST_ARGUMENTS loop
NEW_ARG := MAKE_ARGUMENT_ID ( LX_SYMREP => STORE_SYM ( LIST_ARGUMENTS'IMAGE ( ARG_NAME ) ), XD_POS => LIST_ARGUMENTS'POS ( ARG_NAME )
); NEW_ARG_LIST := APPEND ( NEW_ARG_LIST, NEW_ARG ); end loop; elsif PRAGMA_NAME = OPTIMIZE then for ARG_NAME in OPTIMIZE_ARGU
MENTS loop NEW_ARG := MAKE_ARGUMENT_ID ( LX_SYMREP => STORE_SYM ( OPTIMIZE_ARGUMENTS'IMAGE ( ARG_NAME ) ), XD_POS => OPTIMIZE_ARGUMENTS
'POS ( ARG_NAME ) ); NEW_ARG_LIST := APPEND ( NEW_ARG_LIST, NEW_ARG ); end loop; elsif PRAGMA_NAME = SUPPRESS then for ARG
NAME in SUPPRESS_ARGUMENTS loop NEW_ARG := MAKE_ARGUMENT_ID ( LX_SYMREP => STORE_SYM ( SUPPRESS_ARGUMENTS'IMAGE ( ARG_NAME ) ), XD_POS => SUPPRESS_ARGUMENTS'POS ( ARG_NAME ) ); NEW_ARG_LIST := APPEND ( NEW_ARG_LIST, NEW_ARG ); end loop; elsif PRAGMA_NAME = INTERFA
CE then for ARG_NAME in INTERFACE_ARGUMENTS loop NEW_ARG := MAKE_ARGUMENT_ID ( LX_SYMREP => STORE_SYM ( INTERFACE_ARGUMENTS'IMAGE (
ARG_NAME ) ), XD_POS => INTERFACE_ARGUMENTS'POS ( ARG_NAME ) ); NEW_ARG_LIST := APPEND ( NEW_ARG_LIST, NEW_ARG ); end loop; en
d if; declare SYM: TREE := STORE_SYM ( DEFINED_PRAGMAS'IMAGE ( PRAGMA_NAME ) ); begin NEW_ID := MAKE_PRAGMA_ID ( LX_SYMREP => SYM
, XD_POS => DEFINED_PRAGMAS'POS ( PRAGMA_NAME ), SM_ARGUMENT_ID_S => MAKE_ARGUMENT_ID_S ( LIST => NEW_ARG_LIST ); NEW_ID_LIST := APPEND
( NEW_ID_LIST, NEW_ID ); LIST ( SYM, INSERT ( LIST ( SYM ), NEW_ID ) ); end; for ATTRIBUTE_NAME in DEFINED
_ATTRIBUTES loop declare ITEM_NAME: constant STRING := DEFINED_ATTRIBUTES'IMAGE ( ATTRIBUTE_NAME ); SYM: TREE; begin ITEM_L
LENGTH := ITEM_NAME'LENGTH; if ITEM_NAME ( 1..ITEM_LENGTH ) = "X" then ITEM_LENGTH := ITEM_LENGTH - 2; end if;
SYM := STORE_SYM ( ITEM_NAME ( 1..ITEM_LENGTH ) ); NEW_ID := MAKE_ATTRIBUTE_ID ( LX_SYMREP => SYM, XD_POS => DEFINED_ATTRIBUTES'POS ( ATTR
IBUTE_NAME ) ); NEW_ID_LIST := APPEND ( NEW_ID_LIST, NEW_ID ); PREFIXER LA LISTE DES IDS LIST ( SYM, INSERT ( LIST ( SYM ), NEW
_ID ) ); CHANGER LA XD_DEFLIST PAR-UNE AUGMENTEE EN FIN DE L'ID CRE end; end loop; for OP_NAME in OP_CLASS loop
0150 declare ITEM_NAME: constant STRING := BLTN_TEXT_ARRAY ( OP_NAME ); SYM: TREE; begin ITEM_LENGTH := 3; while ITEM_NAME ( I
TEM_LENGTH ) = '!' loop ITEM_LENGTH := ITEM_LENGTH - 1; end loop; SYM := STORE_SYM ( '!' & ITEM_NAME ( 1..ITEM_LENGTH ) & '!' );
NEW_ID := MAKE_BLTN_OPERATOR_ID ( LX_SYMREP => SYM, SM_OPERATOR => OP_CLASS'POS ( OP_NAME ) ); NEW_ID_LIST := APPEND ( NEW_ID_LIST
, NEW_ID ); PREFIXER LA LISTE DES IDS LIST ( SYM, INSERT ( LIST ( SYM ), NEW_ID ) ); CHANGER LA XD_DEFLIST PAR-UNE AUGMENTEE EN F
IN DE L'ID CRE end; end loop; ID_LIST := NEW_ID_LIST; RENDRE LA LISTE-DES IDS end-MAKE_PREDEF_IDS;
begin declare USER_ROOT: TREE; PREDEF_ID_LIST: SEQ_TYPE; begin USER_ROOT := D ( XD_USER_ROOT, TREE_ROOT ); MAKE_PRED
EF_IDS ( PREDEF_ID_LIST ); NOEUDS STANDARD-POUR LES NOMS PREDEFINIS WALK ( D ( XD_STRUCTURE, USER_ROOT ), PARENT => TREE_VOID, REGIO
N => TREE_VOID ); PARCOURIR L'ARBRE SYNTAXIQUE DU STANDRD declare INTEGER_ID: TREE := HEAD_DEFN ( STORE_SYM ( "INTEGER" ) );
NATURAL_ID: TREE := HEAD_DEFN ( STORE_SYM ( "NATURAL" ) ); POSITIVE_ID: TREE := HEAD_DEFN ( STORE_SYM ( "POSITIVE" ) ); INTGR_SIZE:
INTEGER := DI ( CD_IMPL_SIZE, D ( SM_TYPE_SPEC, INTEGER_ID ) ); DURATION_ID: TREE := HEAD_DEFN ( STORE_SYM ( "DURATION" ) ); DURATION_BASE_I
D: TREE := HEAD_DEFN ( STORE_SYM ( "DURATION" ) ); DURATION_SPEC: TREE := D ( SM_TYPE_SPEC, DURATION_ID ); DURATION_BASE_SPEC: TREE := D
( SM_TYPE_SPEC, DURATION_BASE_ID ); begin DI ( CD_IMPL_SIZE, D ( SM_TYPE_SPEC, NATURAL_ID ), INTGR_SIZE ); DI ( CD_IMPL_SIZE, D ( SM_TYPE_S
PEC, POSITIVE_ID ), INTGR_SIZE ); DI ( CD_IMPL_SIZE, DURATION_BASE_SPEC, DI ( CD_IMPL_SIZE, DURATION_SPEC ) ); D ( SM_BASE_TYPE, DURATION_SPEC
, DURATION_BASE_SPEC ); DB ( SM_IS_ANONYMOUS, DURATION_BASE_SPEC, TRUE ); D ( XD_SOURCE_NAME, DURATION_BASE_SPEC, DURATION_ID ); D (
SM_TYPE_SPEC, D ( SM_RANGE, DURATION_SPEC ), DURATION_BASE_SPEC ); SOUS TYPE CONTRAINTE D'ETENDUE POUR-DURATION end; declare
ADDRESS_ID: TREE := HEAD_DEFN ( STORE_SYM ( "ADDRESS" ) ); BASE_SPEC: TREE := D ( SM_TYPE_SPEC, D ( SM_TYPE_SPEC, ADDRESS_ID ) ); ID DE-TYP
E ANCTRE DANS SM_TYPE_SPEC NEW_SPEC: TREE := COPY_NODE ( BASE_SPEC ); begin D ( XD_SOURCE_NAME, NEW_SPEC, ADDRESS_ID ); D ( SM_DER
IVED, NEW_SPEC, BASE_SPEC ); D ( SM_BASE_TYPE, NEW_SPEC, NEW_SPEC ); D ( SM_TYPE_SPEC, ADDRESS_ID, NEW_SPEC ); end; declare
PACK_SYM: TREE := HEAD_DEFN ( STORE_SYM ( "STANDRD" ) ); CHERCHER LE SYMBOLE NOM DU PACKAGE STANDRD EN VERIFIANT QU'IL A UNE DEFLIST
HEADER: TREE := D ( SM_SPEC, PACK_SYM ); begin LIST ( D ( AS_DECL_S2, HEADER ), PREDEF_ID_LIST ); IDENTIFICATEURS-EN PARTIE-PRIVEE
0170 end; end; end; end FIX_PRE;

```